

Đề cương ôn tập Phân tích thiết kế thuật toán

Dành cho các lớp Trí Tuệ Nhân Tạo & Robot

I. Thời gian 90 phút

II. Hình thức thi viết không sử dụng tài liệu gồm 4 câu

III. Yêu cầu mỗi bài phân tích thuật toán, ví dụ minh họa và code bằng ngôn ngữ C/C++

IV. Nội dung

1. Mọi con đường về 0 (DFS): Một số $n=a*b$ ($a \leq b$) thì sinh ra $m=(a-1)*(b+1)$.

Cho s và f hỏi có tồn tại cách đi từ s đến f

Ý tưởng: Dùng thuật toán DFS và mảng đánh dấu used để xem số được sinh ra đã được duyệt qua chưa. Với mỗi nút $n=a*b$ ta sinh ra các nút con $m=(a-1)*(b+1)$ sao cho $a \leq b$ và $a*a \leq n$.

Thuật toán:

- Sử dụng 1 ngăn xếp
- Dùng 1 mảng đánh dấu những phần tử đã vào stack là 1
- Dùng 1 mảng đánh dấu những phần tử chưa vào stack là 0

Bước 1: đưa s và Stack

Mảng đánh dấu toàn 0 trừ $d[s]=1$

Bước 2: Lấy u ra khỏi Stack

Duyệt các phần tử v sinh ra bởi u :
$$\begin{cases} u = a*b \\ v = (a-1)(b+1) \end{cases}$$

Nếu $d[v]=0$ thì
$$\begin{cases} d[v]=1 \\ stack.push(v) \end{cases}$$

Nếu $v=f \Rightarrow$ return true

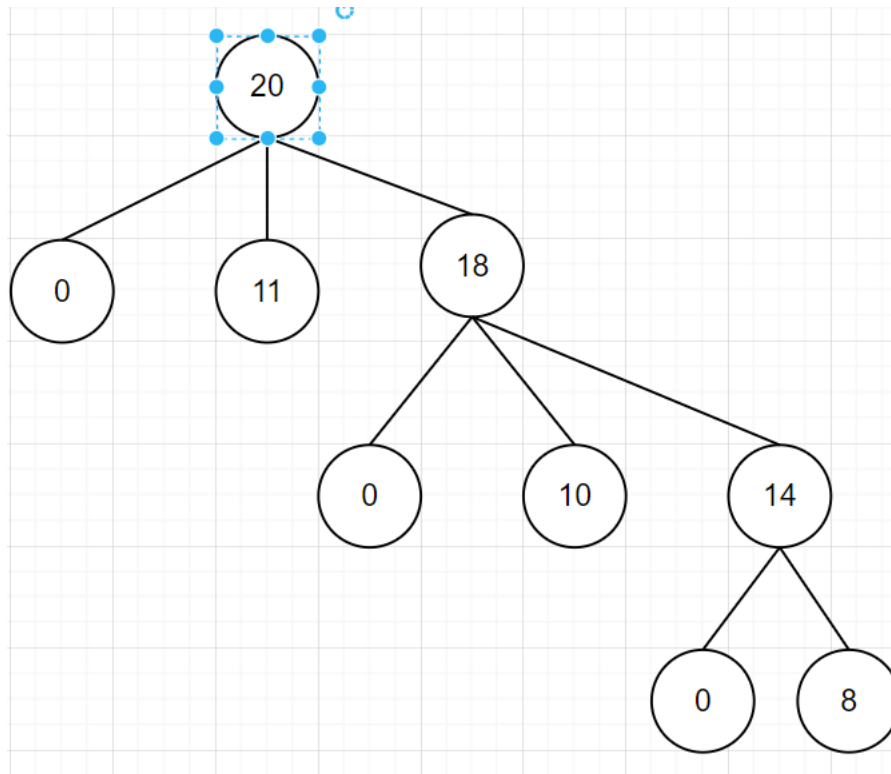
Bước 3: Lặp lại bước 2 tới khi Stack rỗng thì return false

Minh họa: $s=12, f=8$

Stack

20	0	11	18	10	14	8
----	---	----	----	----	----	---

d	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	1								1		1	1			1				1		



2. Bài toán đong nước (BFS): Cho 2 can n, m lít. Muốn đong K lít

Thuật toán :

Sử dụng cấu trúc dữ liệu Queue chứa các phần tử là các cặp pair <int, int>

Sử dụng map <pair <int, int>, int> M để lưu độ sâu tìm kiếm

Bước 1: nhập m, n, k

Bước 2: Q.push({0,0})

M(0,0)=0

Bước 3: Lấy u ra khỏi Q

Đặt x= u.first, y = u.second, t= x+y

Xét Next[]={ {0, e}¹, ... }

Duyệt từng v trong Next

Nếu v chưa có trong M (M.first(v) == M.end(v))

thì
$$\begin{cases} M[v] = M[u] + 1 \\ Q.push(v) \end{cases}$$

Nếu v.first =K hoặc v.second =k thì xuất ra M[v] và stop

Bước 4: Lặp lại bước 3 cho tới khi Q rỗng => không đong được nước

Ví dụ minh họa: n=3, m=5, k=4

3. Phân vùng ảnh (BFS) thuật toán: Cho ma trận ảnh nhị phân 0 và 1. 0 là trắng, 1 là đen. Một vùng trong ảnh đen trắng là một vùng chứa toàn những ô trắng (số 0) sao cho 2 ô bất kỳ trong vùng này liên thông với nhau theo láng giềng 8 và 2 ô ở hai vùng khác nhau thì không tồn tại đường đi nào liên thông láng giềng 8 với nhau theo. Đếm số vùng.

Ý tưởng: Dùng thuật toán DFS. Tìm pixel = 0 đầu tiên của các vùng. Với mỗi pixel ta sẽ duyệt các láng giềng 8 của nó sao cho nếu pixel tiếp theo là 1 thì dừng lại còn nếu là

0 thì tăng số lượng phần tử trong vùng lên và duyệt tiếp các pixel láng giềng của pixel đó.

Thuật toán:

B1: Nhập $A[n][m]$ và mảng d lưu số lượng phần tử của vùng i .

B2: Tạo rào $a[0][j] = a[n+1][j] = 1, a[i][0] = a[i][m+1] = 1$

B3: Duyệt ma trận ảnh, nếu:

+ $a[i][j] = 0$ thì tăng k rồi xét đến các láng giềng của nó và tăng $d[k]$ lên

B4: Xuất ra kết quả.

4. Cách ly covid (BFS): Có n người và m tiếp xúc

Ý tưởng: sử dụng thuật toán BFS. Ta dùng mảng f để lưu xem người đó là thuộc nhóm F mấy và mảng F để lưu số lượng người bị lây nhiễm của nhóm F đó. Với mỗi cư dân, ta sẽ xem những cư dân đó đã tiếp xúc với ai thì đánh dấu cư dân đó là $f+1$ và tăng số lượng người trong F đó lên.

Thuật toán:

Dùng 1 mảng các vectơ $A[n+1]$

Mỗi $A[i]$ là 1 vectơ chứa các đỉnh kề vs i

Dùng Queue chứa các số nguyên để thực hiện BFS

Dùng mảng $d[n+5]$ ban đầu toàn -1 đánh dấu $d[i]$ là i thuộc thể hệ covid F bao nhiêu

Mảng $F[n+5]$ ban đầu toàn 0 đếm số ca nhiễm $F_0, F_1 \dots$ tương ứng

Bước 1: Nhập n, m và m tiếp xúc với x, y

$A[x].push_back(y)$

$A[y].push_back(x)$

Bước 2: Nhập số F_0 và các phần tử F_0 đưa vào Queue đồng thời đánh dấu trên d là 0

Bước 3: Lấy u ra khỏi Queue

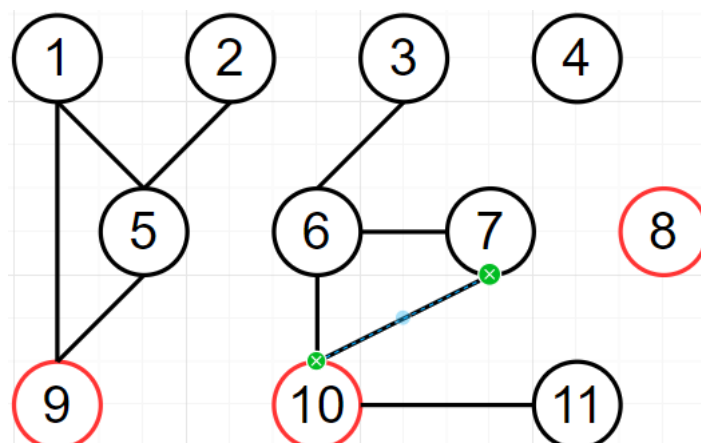
Duyệt v thuộc $A[u]$ nếu $d[v] = -1$ thì

$$\begin{cases} d[v] = d[u] + 1 \\ Q.push(v) \\ F[d[v]]++ \end{cases}$$

Bước 4: Lặp lại bước 3 cho tới khi Q rỗng thì dừng

Bước 5: Duyệt các phần tử F khác 0 và xuất ra màn hình

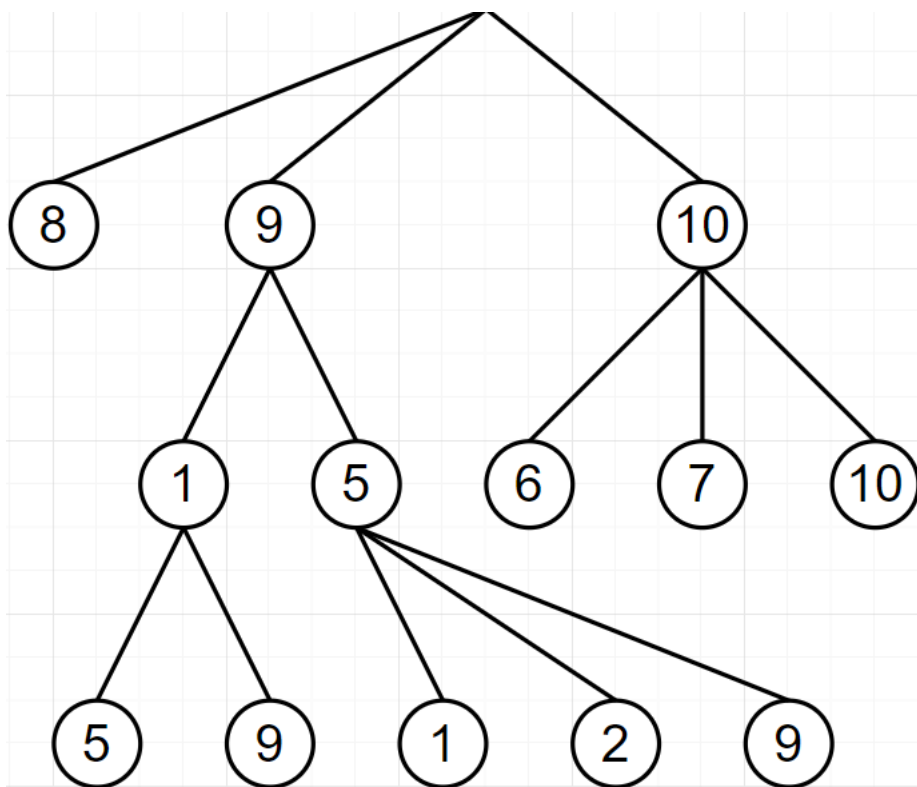
Minh họa: $F_0 = 8, 9, 10$



Q	8	9	10	1	5	6	7	11	2	3
---	---	---	----	---	---	---	---	----	---	---

	1	2	3	4	5	6	7	8	9	10	11
d	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1
	1				1	1	1				1
		2	2								

F	0	1	2
SL	3	5	2



5. **Buôn dưa lê (queue):** Có n mảnh ruộng trồng dưa có số lượng $a_1, a_2, \dots, a_n$ chín lần lượt từ 1 đến n . Dưa từ khi chín thu hoạch trong vòng k ngày. Năng lực thu mỗi ngày là m . hãy thu dưa sao cho tổng số lượng dưa lớn nhất.

Ý tưởng: Sử dụng hàng đợi. Dưa sản lượng từng ngày vào hàng đợi, nếu hôm nay thu hoạch được nhỏ hơn m năng lực thì loại khỏi hàng đợi, ngược lại thì trừ đi m sản lượng. Nếu hàng đợi có lớn hơn k thửa ruộng thì sẽ loại thửa ruộng đầu tiên đi.

Thuật toán:

Sử dụng cấu trúc dữ liệu Queue chứa các số là sản lượng các thửa ruộng

Bước 1: Nhập $n, k, m, s=0$

Bước 2: Duyệt i từ 1 đến $n+k-1$

+ Nhập a_i nếu $i \leq n$ ngược lại $a_i=0$

```

+ Đưa  $a_i$  và Queue
+ Nếu  $Q.size > k$  thì  $Q.pop$ 
+  $t=0$ 
+ while ( $Q.size \&\& Q.front + t \leq m$ ) {
     $t+ = Q.front ; Q.pop$ 
+ if ( $Q.size > 0$ ) {  $Q.front - = m-t; t=m$ }
+  $s+=t$ 

```

Bước 3: xuất s

Minh họa: $n=8, k=3, m=7$

	4	10	2	4	1	19	7	18	1			
S	0	4	7	3+2	4	1	7	7	5+2	5+2	7	1

$$S = 0 + 4 + 7 + 5 + 4 + 1 + 7 + 7 + 7 + 7 + 7 + 1 = 57$$

6. Lập lịch (greedy): Cho n sự kiện, thời gian bắt đầu a, thời gian kết thúc b. Chỉ có 1 nơi để tổ chức. Hãy chọn các sự kiện để tổ chức đc nhiều sự kiện nhất và các sự kiện không đc chồng chéo (có thể giao nhau ở nút)

Ý tưởng: Sắp xếp các sự kiện theo thời gian hòa thành sớm nhất. Khi nào một sự kiện mới có thời gian bắt đầu lớn hơn thời gian kết thúc của sự kiện vừa qua thì ta chọn sự kiện đó.

Thuật toán:

- Dùng pair <int, int> lưu 1 sự kiện
- Dùng mảng lưu trữ dãy các sự kiện

Bước 1: Nhập n và $A_1, \dots, A_n$

Bước 2: Sắp xếp mảng A theo $A_i.second$ tăng dần

Bước 3: $t = -\infty$ (-INT_MAX)

$$S = 0$$

Bước 4: Duyệt từng sự kiện i từ 1 đến n

$$\text{Nếu } A_i.first \geq t \text{ thì } \begin{cases} s++ \\ t = A_i.second \end{cases}$$

Bước 5: Xuất s

Minh họa:

a	4	2	9	8	4	6	8	15	10
b	7	8	12	16	9	10	10	19	14

Bước 1: Sắp xếp theo kết thúc tăng dần

a	4	2	4	8	6	9	10	8	15
b	7	8	9	10	10	12	14	16	19

Bước 2: Tham lam

Bước	a	b	t	Chọn (số sự kiện được chọn)
0			$-\infty$	0
1	4	7	7	1
2	2	8	7	1
3	4	9	7	1
4	8	10	10	2
5	6	10	10	2
6	9	12	10	2
7	10	14	14	3
8	8	16	14	3
9	15	19	19	4

7. Trình thám (priority_queue): Cho n chiều cao và k. Hãy tìm phần tử lớn nhất trong $a_i \dots a_{i+k-1}$

Ý tưởng: Ta dùng hàng đợi ưu tiên lớn, mỗi phần tử là 1 cặp giá trị {chiều cao, vị trí}. Cho ra thằng cao nhất trong hàng đợi và nếu thằng top của hàng đợi nằm ngoài khoảng đang xét thì sẽ loại nó ra.

Thuật toán:

- Sử dụng cấu trúc dữ liệu hàng đợi ưu tiên lớn mỗi phần tử là 1 pair: first: lưu giá trị; second: lưu vị trí

Bước 1: Nhập n, k và khởi tạo Q

Bước 2: Duyệt i từ 1 đến n

Nhập x

Q.push({x, i})

Nếu $i \geq k$ thì while (i-Q.top().second $\geq$ k { Q.pop() }

Xuất Q.top().first

Minh họa: n = 10, k=3

7	4	2	8	3	7	2	1	5	2
		7	8	8	8	7	7	5	5

Bước	Xét	Q	out
0		$\emptyset$	
1	7	{(7,1)}	
2	4	{(7, 1), (4, 2)}	
3	2	{(7, 1), (4, 2), (2,3)}	7

4	8	{(7, 1), (4, 2), (2,3), (8,4)}	8
5	3	{(7, 1), (4, 2), (2,3), (8,4), (3, 5)}	8
6	7	{(7, 1), (4, 2), (2,3), (8,4), (3, 5), (7, 6)}	8
7	2	{(7, 1), (4, 2), (2,3), (8,4), (3, 5), (7, 6), (2, 7)}	7
8	1	{(7, 1), (4, 2), (2,3), (8,4), (3, 5), (7, 6), (2, 7), (1, 8)}	7
9	5	{(7, 1), (4, 2), (2,3), (8,4), (3, 5), (7, 6), (2, 7), (1, 8), (5, 9)}	5
10	2	{(7, 1), (4, 2), (2,3), (8,4), (3, 5), (7, 6), (2, 7), (1, 8), (5, 9), (2, 10)}	5

8. Nối thanh kim loại (priority_queue): Nếu 2 thanh a, b nối lại $\left[\begin{array}{l} \text{dài } a + b \\ \text{chi phí } a + b \end{array} \right.$

Cho n thanh dài $a_1, \dots, a_n$. Hãy nối lại với chi phí ít nhất

Ý tưởng: Sử dụng hàng đợi ưu tiên bé, ta lấy 2 phần tử đầu của hàng đợi ra cộng lại rồi đưa lại vào hàng đợi. Cứ thế cho đến khi hàng đợi còn 1 phần tử.

Thuật toán:

- Sử dụng cấu trúc dữ liệu Hàng đợi ưu tiên bé các số là độ dài các thanh

Bước 1: $Q = \{ \}$, $res = 0$

Bước 2: Nhập n

Bước 3: Nhập $a_1, \dots, a_n$ đưa vào Q

Bước 4: nếu Q có hơn 1 phần tử thì

+ Lấy x, y ra khỏi Q

+ $res += x + y$

+ Đưa $x + y$ vào Q

Bước 5: Lặp lại bước 4 cho tới khi Q còn 1 phần tử

Bước 6: Xuất res

Minh họa:

	4	7	2	8	4	8	3	2
SX	2	2	3	4	4	7	8	8

Bước	PQ	x	y	res
0	{4, 7, 2, 8, 4, 8, 3, 2}			0
1	{4, 7, 8, 4, 8, 3, 4}	2	2	4
2	{7, 8, 4, 8, 4, 7}	3	4	11
3	{7, 8, 8, 7, 8}	4	4	19
4	{8, 8, 8, 14}	7	7	33

5	{8, 14, 16}	8	8	49
6	{16, 22}	8	14	71
7	{38}	16	22	109

9. Phần tử trung vị (priority_queue): Trung vị là phần tử ở vị trí $n/2$ sau khi sắp xếp.

Cho $a_1, \dots, a_n$. Tìm trung vị: $a_1, a_1 a_2, a_1 \dots a_n$

Y tưởng: Sử dụng 2 hàng đợi ưu tiên L và R, L: ưu tiên lớn, R: ưu tiên bé. Lúc đó ta sẽ cho các phần tử vào 2 hàng đợi sao cho mọi phần tử của L phải nhỏ hơn phần tử top() của R thì ta sẽ tìm được trung vị là L.top().

Thuật toán:

- Sử dụng 2 hàng đợi ưu tiên chứa các số nguyên trong đó L: ưu tiên lớn, R: ưu tiên bé

Bước 1: Nhập n

Bước 2: Duyệt i từ 1 đến n

+ Nhập x

+ Nếu i lẻ: L.push(x)

+ Nếu i chẵn: R.push(x)

+ Nếu $i \geq 2$ & L.top() > R.top() thì lấy 2 phần tử top ở 2 bên ra và đổi bên cho nhau

+ Xuất L.top()

Minh họa:

	7	4	2	1	8	0	7	2
Trung vị	7	4	4	4	7	7	7	7

Bước	xét	L	R	out
0				
1	7	{7}		7
2	4	{7}	{4}	
đổi top		{4}	{7}	4
3	2	{4, 2}	{7}	4
4	1	{4, 2}	{7, 1}	
đổi top		{1, 2}	{7, 4}	2
5	8	{1, 2, 8}	{7, 4}	
đổi top		{1, 2, 4}	{7, 8}	4
6	0	{1, 2, 4}	{8, 7, 0}	

đổi top		{1, 2, 0}	{8, 7, 4}	2
7	7	{0, 1, 2, 7}	{8, 7, 4}	
đổi top		{0, 1, 2, 4}	{8, 7, 7}	4
8	2	{0, 1, 2, 4}	{7, 7, 8, 2}	
đổi top		{0, 1, 2, 2}	{7, 7, 8, 4}	2

10. Quân hậu (Backtracking): Cho bàn cờ nxn hãy tìm cách đặt n quân hậu sao cho không con nào ăn nhau

Ý tưởng: Dùng thuật toán quay lui. Với mỗi con hậu ở hàng thứ i ($i \leq n$), ta sẽ đặt nó vào các cột j ($j \leq n$) sao cho con hậu đó không được nằm trên cùng một hàng hoặc một cột hoặc một đường chéo chính hoặc một đường chéo phụ với một con đã được đặt vào trước.

Thuật toán:

B1: Nhập n

B2: Gọi đến hàm TRY bắt đầu từ hàng thứ 1.

B3: Trong hàm TRY:

Ở hàng thứ i ta duyệt lần lượt các cột thứ j :

+ Nếu ở vị trí $a[i][j]$ mà chưa có con nào xuất hiện cùng hàng hoặc cùng cột hoặc đường chéo chính hoặc đường chéo phụ thì ta đánh dấu quân cờ ở đó và gọi hàm TRY đến hàng thứ $i+1$.

+ Nếu duyệt đến hàng cuối cùng thì ta cập nhật kết quả.

B4: Xuất kết quả

Minh họa: bàn cờ 4x4

	X						X	
			X		X			
X								X
		X				X		

11. Đổi tiền (Quy hoạch động): Cho n mệnh giá $a_1 \dots a_n$ và số tiền M . Tìm cách đổi số tờ ít nhất

Ý tưởng: Đặt C_{ij} là số tờ ít nhất khi dùng $a_1 a_2 \dots a_i$ đổi ra j tiền

Ta có: $C_{i0} = 0$ $C_{0j} = \infty$

Nếu $a_i > j$ thì $C_{ij} = C_{(i-1)j}$

Nếu $a_i < j$ thì $C_{ij} = \min(C_{(i-1)j} + C_{i(j-a_i)})$

Thuật toán:

Bước 1: Nhập $n, M, a_1 \dots a_n$

Bước 2: Khai báo mảng 2 chiều C có $(n+1)$ hàng $(M+1)$ cột

Bước 3: Duyệt i từ 0 đến n , $C_{i0} = 0$

Bước 4: Duyệt j từ 1 đến M , $C_{0j} = M+1$

Bước 5: Duyệt i từ 0 đến n

Duyệt j từ 1 đến M

Nếu $a_i > j$ thì $C_{ij} = C_{(i-1)j}$

Ngược lại, $C_{ij} = \min(C_{(i-1)j} + C_{i(j-a_i)})$

Bước 6: Xuất C_nM

Minh họa: $n=4, M=24, 5\ 2\ 7\ 4$

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
*	0	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞
5	0	∞	∞	∞	∞	1	∞	∞	∞	∞	2	∞	∞	∞	∞	3	∞	∞	∞	∞	∞	∞	∞	∞	∞
2	0	∞	1	∞	2	1	3	2	4	3	2	4	3	5	4	3	5	4	6	5	4	6	5	7	6
7	0	∞	1	∞	2	1	3	1	4	2	2	3	2	4	2	3	3	3	4	3	4	3	4	4	4
4	0	∞	1	∞	1	1	2	1	2	2	2	2	2	3	2	3	3	3	3	3	4	3	4	4	4

Truy vết: Số tờ ít nhất là 4: $7+7+5+5$

12. Sắp xếp balo (Quy hoạch động): Cho n đồ vật có kích thước $a_1... a_n$ có giá trị $b_1... b_n$. Cho kích thước balo M. Tìm cách lấy các vật bỏ vào trong balo sao cho tổng kích thước $\leq M$ và tổng giá trị là lớn nhất

Ý tưởng: Đặt C_{ij} là tổng giá trị lớn nhất. Chọn các đồ vật từ 1 đến i xếp vào balo kích thước j

Ta có: $C_{i0} = 0$ (kích thước balo =0)

$C_{0j} = \infty$ (không có đồ vật)

Nếu $a_i > j$ thì $C_{ij} = C_{(i-1)j}$

Nếu $a_i \leq j$ thì $C_{ij} = \max(C_{(i-1)j}, b_i + C_{(i-1)(j-a_i)})$

$C_{ij} = \min(\min(C_{kj} + C_{i-kj}), \min(C_{ik} + C_{i,j-k}))$

Thuật toán:

Bước 1: Nhập n, M, $(a_1, b_1) \dots (a_n, b_n)$

Bước 2: Khởi tạo $C[n+1][M+1] = \{ \}$

Bước 3: Duyệt i từ 0 đến n

Duyệt j từ 1 đến M

Nếu $a_i > j$ thì $C_{ij} = C_{(i-1)j}$

Ngược lại, $C_{ij} = \max(C_{(i-1)j}, b_i + C_{(i-1)(j-a_i)})$

Bước 4: Xuất C_nM

Minh họa: $M=14$

A	4	2	4	3	5	2
B	7	5	8	7	11	4

C	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
---	---	---	---	---	---	---	---	---	---	---	----	----	----	----	----

b	a	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
7	4	0	0	0	0	7	7	7	7	7	7	7	7	7	7	7
5	2	0	0	5	5	7	7	12	12	12	12	12	12	12	12	12
8	4	0	0	5	5	8	8	13	13	15	15	20	20	20	20	20
7	3	0	0	5	7	7	12	13	15	15	20	20	22	22	27	27
11	5	0	0	5	7	7	12	13	16	18	20	23	24	26	27	31
4	2	0	0	5	7	9	12	13	16	18	20	23	24	27	28	31

Tổng giá trị: 31

Truy vết: $11+7+8+5=31$

13. Cắt hình vuông ít nhất (Quy hoạch động): Cho hình chữ nhật cỡ $n \times m$. Mỗi nhất cắt 1 hình chữ nhật thành 2 hình vuông. Tìm số hình vuông ít nhất.

Ý tưởng: Đặt C_{ij} là số hình vuông ít nhất khi cắt hình $i \times j$

Ta có: $C_{1j} = j$ $C_{i1} = i$ $C_{ii} = 1$

$C_{ij} = \min(\min(C_{kj} + C_{i-kj}), \min(C_{ik} + C_{i,j-k}))$

Minh họa: $n=8, m=11$

C	1	2	3	4	5	6	7	8	9	10	11
1	1^1	2^{-1}	3^{-1}	4^{-1}	5^{-1}	6^{-1}	7^{-1}	8^{-1}	9^{-1}	10^{-1}	11^{-1}
2	2^1	1	3^{-1}	2^{-2}	4^{-1}	3^{-2}	5^{-1}	4^{-2}	6^{-1}	5^{-2}	7^{-1}
3	3^1	3^1	1	4^{-1}	4^{-2}	2^{-3}	5^{-1}	5^{-2}	3^{-3}	6^{-1}	6^{-2}
4	4^1	2^2	4^1	1	5^{-1}	3^{-2}	5^{-3}	2^{-4}	6^{-1}	4^{-2}	6^{-3}
5	5^1	4^1	4^2	5^1	1	5^2	5^{-2}	5^{-3}	6^{-4}	2^{-5}	6^{-5}
6	6^1	3^2	2^3	3^2	5^{-2}	1	5^{-3}	4^{-2}	3^{-3}	4^{-4}	6^{-2}
7	7^1	5^1	5^1	5^3	5^2	5^3	1	7^3	6^{-2}	6^{-3}	6^{-4}
8	8^1	4^2	5^2	2^4	5^3	4^2	7^{-3}	1	7^{-4}	5^{-2}	6^{-3}

14. Xâu con chung dài nhất (Quy hoạch động): Cho xâu $x=x_1 x_2 \dots x_n$ và $y=y_1 y_2 \dots y_n$.

Tìm xâu con chung dài nhất của 2 xâu

Thuật toán:

Bước 1: Nhập $n, a_1, \dots, a_n$

Bước 2: Khai báo C

Bước 3: Duyệt i từ 1 đến n

+ $C_i = 0$

+ Duyệt $j=1$ đến $i-1$

Nếu $a_j > a_i$ & $C_j > C_i$ thì $C_j = C_i$

+ $C_i ++$

Bước 4: Xuất Max ($C_1, \dots, C_n$)

Minh họa: $x=AGXXTAGA, y=XATAGXXG$

C	★	X	A	T	A	G	X	X	G
---	---	---	---	---	---	---	---	---	---

★	0	0	0	0	0	0	0	0	0
A	0	0	1	1	1	1	1	1	1
G	0	0	1	1	1	2	2	2	2
X	0	1	1	1	1	2	3	3	3
X	0	1	1	1	1	2	3	4	4
T	0	1	1	2	2	2	3	4	4
A	0	1	2	2	3	3	3	4	4
G	0	1	2	2	3	4	4	4	5
A	0	1	2	2	3	4	4	4	5

15. Truy vấn max (cây IT): Cho $a_1, \dots, a_n$ và m truy vấn. Mỗi truy vấn L, R tìm max ($a_L \dots a_R$)

Ý tưởng: Ta xây dựng cây IT theo phương pháp chia để trị sao cho mỗi nút là giá trị max của dãy con trong khoảng $[a, b)$ và cây IT là một cây nhị phân với các nút con giá trị trên khoảng $[a, (a+b)/2)$ và $[(a+b)/2, b)$.

Thuật toán:

B1: Tạo cây với mảng Tree toàn giá trị $-\text{INT_MAX}$

B2: Update

- Với một giá trị mới đưa vào:

+ Nếu nó lớn hơn giá trị nút gốc thì cập nhật giá trị nút gốc mới.

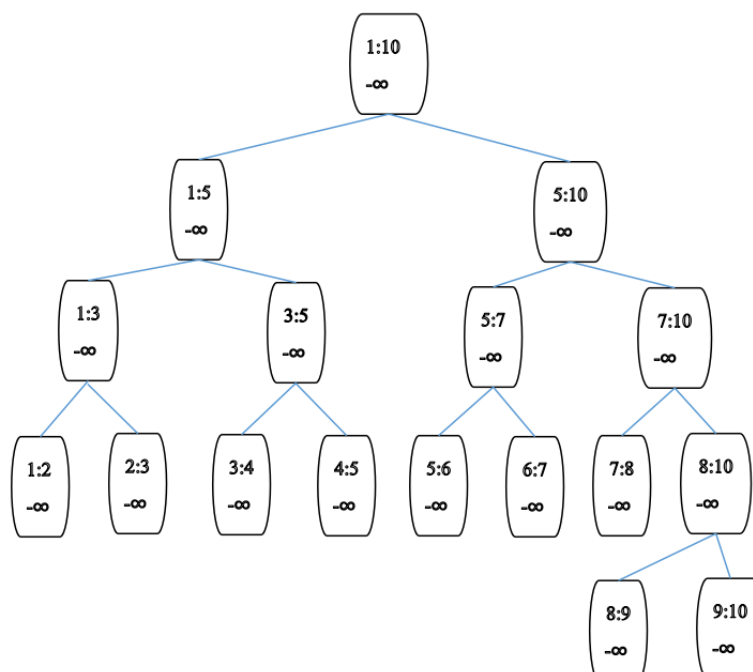
+ Nếu vị trí của giá trị trong dãy số nhỏ hơn $n/2$ thì ta cập nhật các nút con bên trái nút gốc, ngược lại thì cập nhật các nút con bên phải nút gốc.

B3: Thực hiện truy vấn khoảng L đến R .

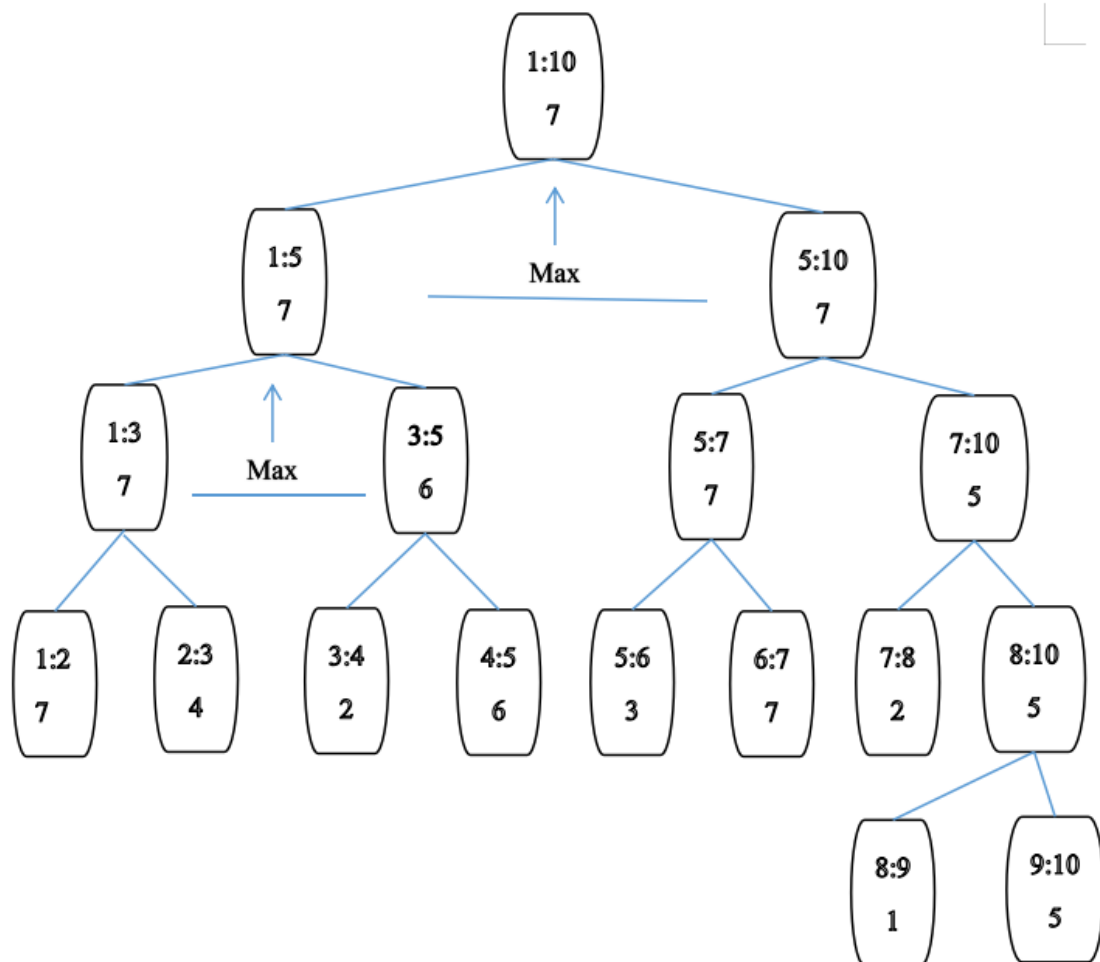
Minh họa: $n=9, m=4$, 7 4 2 6 3 7 2 1 5

$L=2, R=5 \Rightarrow 7$

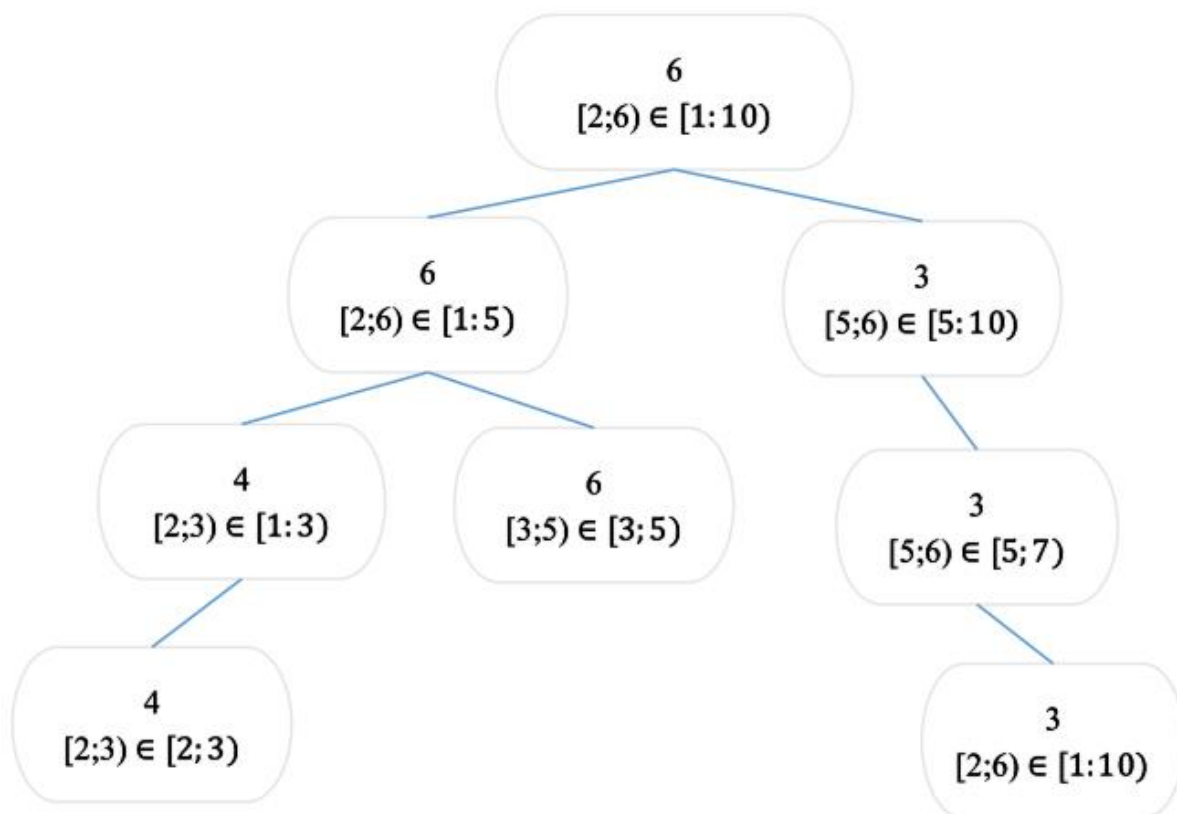
Bước 1: Tạo cây



Bước 2: Update



Bước 3: Truy vấn

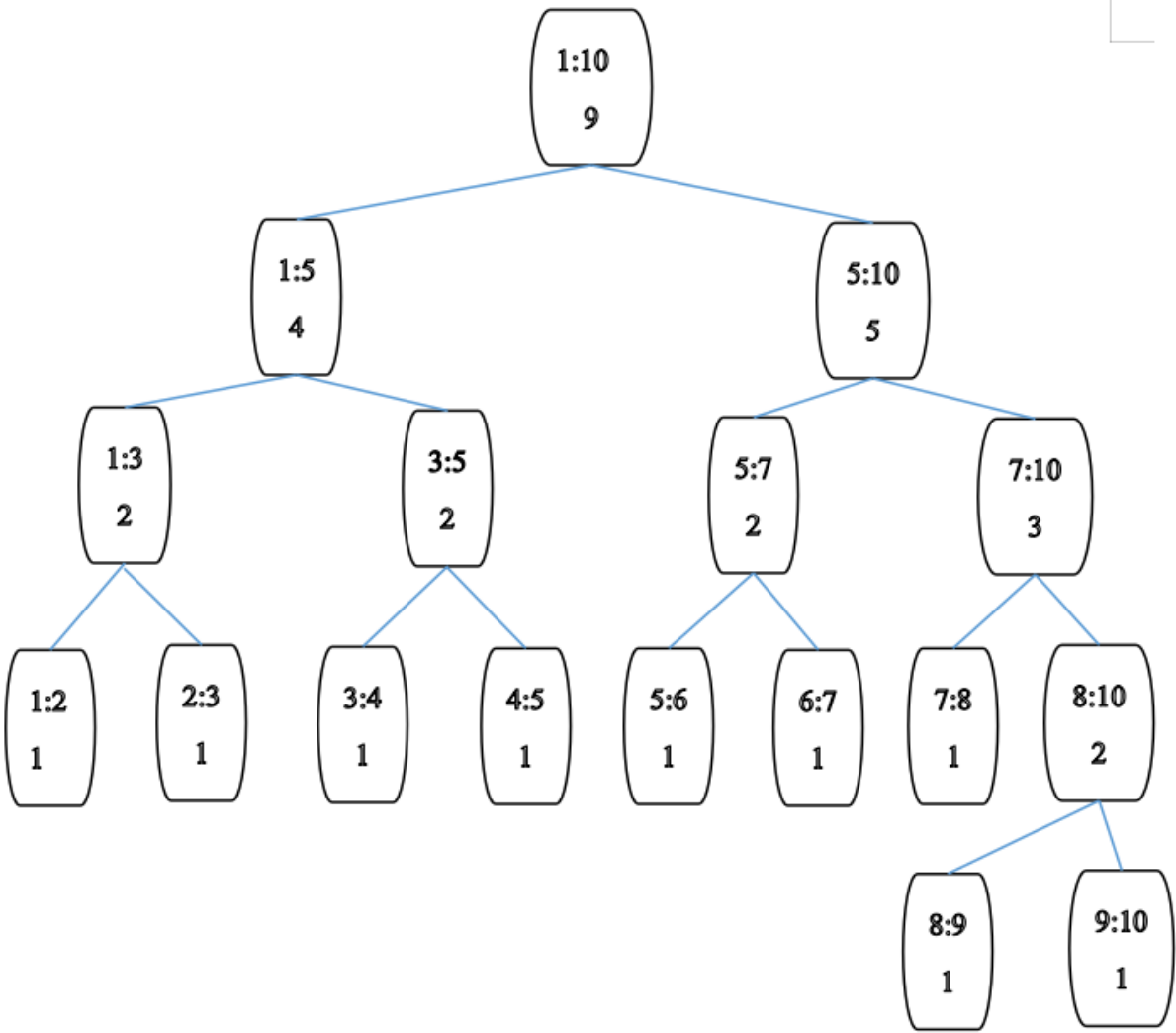


16. Đếm số nghịch thế (Cây IT): Cho 1 cặp hoán vị của tập {1, n}. Một cặp là nghịch thế nếu: $i < j$ và $a_i > a_j$

Minh họa:

7	4	2	8	1	6	9	5	3
0	1	2	0	4	2	0	4	6

Bước 1: Tạo cây



Bước 2: Update and get

Bước 3:

Bước	Xét	Find	res
0			0
1	7	$R \Rightarrow R \Rightarrow L(0)$	0
2	4	$L(1) \Rightarrow R \Rightarrow R$	1
3	2	$L(1) \Rightarrow L(1) \Rightarrow R$	2
4	8	$R \Rightarrow R \Rightarrow R \Rightarrow L(0)$	0
5	1	$L(2) \Rightarrow L(1) \Rightarrow L(1)$	4

6	6	$R \Rightarrow L(2) \Rightarrow R$	2
7	9	$R \Rightarrow R \Rightarrow R \Rightarrow R$	0
8	5	$R \Rightarrow L(3) \Rightarrow L(1)$	4
9	3	$L(5) \Rightarrow R \Rightarrow L(1)$	6
			19